# beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

**Abstract**

This package allows the management of multiple named overlay specifications in beamer documents. Named overlay specifications are very handy both during edition and to manage complex and variable beamer overlay specifications. In particular, they allow to replace raw numbers in beamer <...> overlay specifications by logical identifiers. Demonstration files are available for download as part of the development repository. As a side effect, this is another solution to this latex.org forum query.

## Contents

# 1 Implementation

## 1.1 Package declarations

```
\NeedsTeXFormat{LaTeX2e}[2025/01/01]
\ProvidesExplPackage
  {beanoves}
  {2025/06/03}
  {1.0}
  {Named overlay specifications for beamer}
```

## 1.2 Overview

Reserved namespace: identifiers containing the case insensitive string `beanoves` or containing the case insensitive string `bnvs` delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into beamer specifications based on this data model.

The development is made easier due to a facility layer over expl.

## 1.3 Facility layer: definitions and naming

GOBBLED

## 1.4 Debug facilities

GOBBLED

## 1.5 Local variables

GOBBLED

## 1.6 Infinite loop management

GOBBLED

## 1.7 Overlay specification

### 1.7.1 Registration

GOBBLED

### 1.7.2 Loops

GOBBLED

### 1.7.3 Path function

GOBBLED

### 1.7.4 Low level IP model functions

### 1.7.5 Low level I model functions

### 1.7.6 Getting

GOBBLED

### 1.7.7 Circular dependencies

GOBBLED

### 1.7.8 Spring cleaning

GOBBLED

### 1.7.9 The provide mode

GOBBLED

### 1.7.10 Initialize

GOBBLED

## 1.8 Regular expressions

GOBBLED

```
\regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
  \A \s* (?:
```

- 2 → V

```
    ( [^:]+? )
```

- 3, 4, 5 → A : Z? or A :: L?

```
    | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
```

- 6, 7 → ::(L:Z)?

```
    | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
```

- 8, 9 → :(Z::L)?

```
    | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
  )
  \s* \Z
}
```

GOBBLED

## 1.9 End group setters

GOBBLED

## 1.10 **beamer.cls** interface

GOBBLED

## 1.11 Utilities

### 1.11.1 Split utilities

GOBBLED

### 1.11.2 Match utilities

GOBBLED

## 1.12 Main scanner

GOBBLED

### 1.12.1 Base grammar

GOBBLED

### 1.12.2 Storage

GOBBLED

### 1.12.3 AST exportation policy

This is used by \BNVS_end_scan: to wrap the AST created during the scanning process before it is exported. The case of an empty created AST is sometimes special.

\BNVS_ast_prefix_set:n \BNVS_ast_prefix_set:n {⟨prefix:w⟩}
\BNVS_ast_prefix_will_set:n \BNVS_ast_prefix_will_set:n {⟨prefix:w⟩}

Set the function \BNVS_ast_prefix:w to expand to \exp_not:n { ⟨prefix:w⟩}. This is used to define \BNVS_ast_plc:n, when the arguments must be scanned prior to knowing which function ⟨prefix:w⟩ fits. The "will" variant defers the execution just before \BNVS_end:.

```
\cs_new:Npn \BNVS_ast_prefix:w {}

\cs_new:Npn \BNVS_ast_prefix_set:n #1 {
  \cs_set:Npn \BNVS_ast_prefix:w { \exp_not:n { #1 } }
}

\cs_new:Npn \BNVS_ast_prefix_will_set:n #1 {
  \BNVS_will_end:n {
    \BNVS_ast_prefix_set:n { #1 }
  }
}
```

\BNVS_ast_plc_wrap:n \BNVS_ast_plc_wrap:n {⟨template:n⟩}
\BNVS_ast_plc_wrap_pfx:n \BNVS_ast_plc_wrap_pfx:n {⟨template:n⟩}
\BNVS_ast_plc_wrap_mpt:n \BNVS_ast_plc_wrap_grp:n {⟨template:n⟩}
\BNVS_ast_plc_wrap_grp:n \BNVS_ast_plc_wrap_mpt:n {⟨template:n⟩}
\BNVS_ast_plc_Raw: \BNVS_ast_plc_Raw

Set the AST exportation policy through the meaning of the function \BNVS_ast_plc:n. The ⟨template:n⟩ is by default ##1. With the first variant, nothing is exported if the input AST is blank. With the pfx variant, only the ⟨prefiX:w⟩ is exported when the input AST is blank. The mpt and grp variants never export an empty AST. The first one exports the input AST as is whereas the second one exports the input AST between a pair of braces. \BNVS_ast_plc_Raw exports as is, with no brace nor prefix.

```
\cs_new:Npn \BNVS_ast_plc_wrap:n #1 {
  \cs_set:Npn \BNVS_ast_plc:n ##1 {
    \tl_if_blank:nF { ##1 } {
      \BNVS_ast_prefix:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
      \exp_not:n { #1 }
    }
  }
}
\BNVS_ast_plc_wrap:n { #1 }

\cs_new:Npn \BNVS_ast_plc_wrap_pfx:n #1 {
  \cs_set:Npn \BNVS_ast_plc:n ##1 {
    \BNVS_ast_prefix:w
    \tl_if_blank:nF { ##1 } {
```

and \exp_not:n{⟨xprt:n⟩}.

```
      \exp_not:n { #1 }
    }
  }
}

\cs_new:Npn \BNVS_ast_plc_wrap_mpt:n #1 {
s
  \cs_set:Npn \BNVS_ast_plc:n ##1 {
    \BNVS_ast_prefix:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
    \exp_not:n { #1 }
  }
}

\cs_new:Npn \BNVS_ast_plc_wrap_grp:n #1 {
s
  \cs_set:Npn \BNVS_ast_plc:n ##1 {
    \BNVS_ast_prefix:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
    \exp_not:n { { #1 } }
  }
}

\cs_new:Npn \BNVS_ast_plc_Raw: {
s
  \cs_set:Npn \BNVS_ast_plc:n ##1 { \exp_not:n { ##1 } }
}
```

### 1.12.4 Branch functions

The scanning process only applies to characters of category codes space and other. It definitely stops at the first token that is not of that category. It takes one of 6 branches depending on what is scanned:

- ⟨*Block:w*⟩ for unit blocks like names,
- ⟨*One_colon:w*⟩ after a single colon,
- ⟨*Colons:w*⟩ after multiple colons,
- ⟨*comMa:w*⟩ after a comma,
- ⟨*cLose:w*⟩ after a closing delimiter,
- ⟨*Stop:*⟩ when scanning is done.

**Block branch** The Block branch is used when the last character scanned from the ⟨*input stream*⟩ fits the current requirement as defined by the last scan function called, but the next one does not.

| | |
|---|---|
| \BNVS_scan_Block:Fw | \BNVS_scan_Block:Fw {⟨*else:w*⟩} ⟨*input stream*⟩ |
| \BNVS_scanned_Block:w | \BNVS_scanned_Block:w ⟨*input stream*⟩ |
| \BNVS_scanned_Block_set:n | \BNVS_scanned_Block_set:n {⟨*Block:w*⟩} |
| \BNVS_scanned_Block_wrap::n | \BNVS_scanned_Block_wrap::n {⟨*scAn:B:w*⟩} |
| \BNVS_scan_Block:TFw | \BNVS_scan_Block:TFw {⟨*yes:w*⟩} {⟨*else:w*⟩} ⟨*input stream*⟩ |
| \BNVS_scan_Block_def:n | \BNVS_scan_Block_def:n {⟨*block:TFw*⟩} |

The \BNVS_scanned_Block:w branch function is called by \BNVS_scan_Block:Fw once a ⟨*block*⟩ has been scanned from the head of the ⟨*input stream*⟩ (greedy match). The function \BNVS_scanned_Block_set:n sets the meaning of \BNVS_scanned_Block:w to ⟨*Block:w*⟩. The exact definition of a block is driven by \BNVS_scan_Block:TFw which meaning is set by \BNVS_scan_Block_def:n.

The \BNVS_scanned_Block_wrap::n is a wrapper over \BNVS_scanned_Block_set:n. Its argument is the body of a function with one argument which is the meaning of current \BNVS_scanned_Block:w at the time of the call.

```
\cs_new:Npn \BNVS_scan_Block:Fw #1 {
  \BNVS_scan_Block:TFw {
    \BNVS_scanned_Block:w
  } {
    #1
  }
```

There must be absolutely no code here. No hooking allowed either.

```
}

\cs_new:Npn \BNVS_scan_Block_def:n #1 {
  \cs_set:Npn \BNVS_scan_Block:TFw ##1 ##2 {
    #1
  }
}

\cs_new:Npn \BNVS_scanned_Block_wrap_do:n #1 {
  \BNVS_scanned_Block_set:n {
    \BNVS_scanned_Block_wrapper:B { #1 }
  }
}

\cs_new:Npn \BNVS_scanned_Block_wrap::n #1 {
  \cs_set:Npn \BNVS_scanned_Block_wrapper:B ##1 { #1 }
  \exp_args:No \BNVS_scanned_Block_wrap_do:n { \BNVS_scanned_Block:w }
}

\BNVS_scan_Block_def:n {
```

At the top level, the ⟨*else:w*⟩ instructions are always executed.

```
  #2
}
```

### Colon branch

\c__bnvs_Colons_regex For ranges defined by a colon syntax. One catching group for more than one colon.

```
\regex_const:Nn \c__bnvs_Colons_regex { \s* :(:*) \s* }
```

(*End of definition for* `\c__bnvs_Colons_regex`.)

| | |
|---|---|
| `\BNVS_scan_colon:Fw` | `\BNVS_scan_colon:Fw {`⟨*else:w*⟩`}` ⟨*input stream*⟩ |
| `\BNVS_scanned_One_colon:w` | `\BNVS_scanned_One_colon:w` ⟨*input stream*⟩ |
| `\BNVS_scanned_One_colon_set:n` | `\BNVS_scanned_One_colon_set:n {`⟨*One_colon:w*⟩`}` |
| `\BNVS_scanned_Colons:w` | `\BNVS_scanned_Colons:w` ⟨*input stream*⟩ |
| `\BNVS_scanned_Colons_set:n` | `\BNVS_scanned_Colons_set:n {`⟨*Colons:w*⟩`}` |

On scanning a standalone colon from the ⟨*input stream*⟩, calls the branch function `\BNVS_scanned_One_colon:w`. On scanning multiple colons from the ⟨*input stream*⟩, calls the `\BNVS_scanned_Colons:w` branch function. In other cases, execute the ⟨*else:w*⟩ instructions.

One shot utility function only used by `\BNVS_scan_colon:Fw`

```
\cs_new:Npn \BNVS_scanned_Colons_regex:n #1 {
  \tl_if_empty:nTF { #1 } {
    \BNVS_scanned_One_colon:w
  } {
    \BNVS_scanned_Colons:w
  }
}

\cs_new:Npn \BNVS_scan_colon:Fw #1 {
  \peek_regex_replace_once:NnTF \c__bnvs_Colons_regex { \cB\{ \1 \cE\} } {
    \BNVS_scanned_Colons_regex:n
  } {
    #1
  }
}
```

### Comma branch

\c__bnvs_comMa_regex Comma as a separator.

```
\regex_const:Nn \c__bnvs_comMa_regex { \s* , \s* }
```

(*End of definition for* `\c__bnvs_comMa_regex`.)

`\peek_regex_remove_once:NTF` is buggy in L3 programming layer <2025-01-18>.

```
\cs_new:Npn \BNVS_peek_regex_remove_once:NTF #1 {
  \peek_regex_replace_once:NnTF #1 { }
}
```

| | |
|---|---|
| `\BNVS_scan_comMa:Fw` | `\BNVS_scan_comMa:Fw {`⟨*else:w*⟩`}` ⟨*input stream*⟩ |
| `\BNVS_scanned_comMa:w` | `\BNVS_scan_comMa:w` ⟨*input stream*⟩ |
| `\BNVS_scanned_comMa_set:n` | `\BNVS_scan_comMa_set:n {`⟨*comMa:w*⟩`}` |

Once a comma has just been scanned from the head of the ⟨*input stream*⟩, calls `\BNVS_scanned_comMa:w` branch function, otherwise execute ⟨*else:w*⟩ instructions. `\BNVS_scan_comMa_set:n` sets the meaning of `\BNVS_scan_comMa:w` to ⟨*comMa:w*⟩.

```
\cs_new:Npn \BNVS_scan_comMa:Fw #1 {
```

\peek_regex_remove:NnTF is buggy in L3 programming layer <2025-01-18>.

```
  \BNVS_peek_regex_remove_once:NTF \c__bnvs_comMa_regex {
    \BNVS_scanned_comMa:w
  } {
    #1
  }
```

There must be absolutely no code here. No hooking allowed either.

```
}
```

### Close branch

\c__bnvs_cLose_regex

Closing delimiters.

```
\regex_const:Nn \c__bnvs_cLose_regex { \s* (%([{
  \}|\]|\)
  ) \s*
}
```

(*End of definition for* \c__bnvs_cLose_regex.)

\BNVS_scan_cLose:Fw `\BNVS_scan_cLose:Fw {`⟨*else:w*⟩`}` ⟨*input stream*⟩
\BNVS_scanned_cLose:nw `\BNVS_scanned_cLose:nw` ⟨*clositer*⟩ ⟨*input stream*⟩
\BNVS_scanned_cLose:w `\BNVS_scanned_cLose:w` ⟨*input stream*⟩
\BNVS_scanned_cLose_set: `\BNVS_scanned_cLose_set:n` ⟨*cLose:w*⟩

When the \BNVS_scan_cLose:Fw function scans one of '`)`', '`]`' or '`}`' closing delimiter, it calls the branch function \BNVS_scanned_cLose:nw with that delimiter as argument, which in turn calls \BNVS_scanned_cLose:w, after it manages eventual errors in balancing delimiters. Otherwise execute ⟨*else:w*⟩ code.

```
\cs_new:Npn \BNVS_scan_cLose:Fw #1 {
  \peek_regex_replace_once:NnTF \c__bnvs_cLose_regex { \cB\{ \1 \cE\} } {
    \BNVS_scanned_cLose:nw
  } {
    #1
  }
}
\cs_new:Npn \BNVS_scanned_cLose:nw #1 {
  \BNVS_error:n { Unexpected~delimiter~`#1' }
  \BNVS_scanned_cLose:w
}
\tl_map_inline:nn {{Block}{One_colon}{Colons}{comMa}{cLose}} {
  \cs_new:cpn { BNVS_scanned_#1_set:n } ##1 {
    \cs_set:cpn { BNVS_scanned_#1:w } {
      ##1
    }
  }
```

Do nothing operation at the top level.

```
  \cs_new:cpn { BNVS_scanned_#1:w } {
  }
}
```

**Stop branch** The Stop branch is executed in the Stop situation, *id est* when the head of the ⟨*input stream*⟩ cannot be scanned. In practice, when the head of the ⟨*input stream*⟩ is not a character of category code space or other. GOBBLED

### 1.12.5 Begin and end of scanning processes

GOBBLED

### 1.12.6 Main scan functions

GOBBLED

### 1.12.7 ⟨*Rnd*⟩, ⟨*Sqr*⟩ and ⟨*Crl*⟩ units

We assume that delimiters are paired and balanced. Nevertheless, some errors are catched and properly managed. Here is a grammar for material enclosed between delimiters

```
⟨Rnd(⟨blnc⟩)⟩  ::= ⟨Rnd_open⟩  ⟨blnc⟩  ⟨Rnd_close⟩
⟨Rnd_open⟩     ::= ⟨spc⟩  '('  ⟨spc⟩
⟨Rnd_close⟩    ::= ⟨spc⟩  ')'  ⟨spc⟩
⟨Sqr(⟨blnc⟩)⟩  ::= ⟨Sqr_open⟩  ⟨blnc⟩  ⟨Sqr_close⟩
⟨Sqr_open⟩     ::= ⟨spc⟩  '['  ⟨spc⟩
⟨Sqr_close⟩    ::= ⟨spc⟩  ']'  ⟨spc⟩
⟨Crl(⟨blnc⟩)⟩  ::= ⟨Crl_open⟩  ⟨blnc⟩  ⟨Crl_close⟩
⟨Crl_open⟩     ::= ⟨spc⟩  '{'  ⟨spc⟩
⟨Crl_close⟩    ::= ⟨spc⟩  '}'  ⟨spc⟩
```

with different variations of associate AST's. (missing description for ⟨*Sqr*⟩ and ⟨*Crl*⟩ are similar)

```
⟨Rnd(⟨blnc⟩)⟩*::= ⟨prefix:w⟩ { ⟨blnc⟩* }
                |  ⟨prefix:w⟩   ⟨blnc⟩*
                |              { ⟨blnc⟩* }
                |                ⟨blnc⟩*
```

The exported AST also depends on the exportation policy, whether ⟨*blnc*⟩ is empty or not.

\BNVS_scan_cLose:N:EbsTN:w
\BNVS_scan_cLose:N:PXbsTN:w

```
\BNVS_scan_cLose:N:EBSTN:w ⟨cLose⟩
  {⟨prEamble:⟩}           {⟨scAn:B:w⟩} {⟨scAn:S:⟩} {⟨posTamble:⟩} {⟨scAn:w⟩}
\BNVS_scan_cLose:N:PXBSTN:w ⟨cLose⟩
  {⟨xprt:n⟩} {⟨prefix:w⟩} {⟨scAn:B:w⟩} {⟨scAn:S:⟩} {⟨posTamble:⟩} {⟨scAn:w⟩}
```

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter using the ⟨*scAn:w*⟩ instructions. Implementation detail: we begin a scanning group that will be closed on scanning the ⟨*close*⟩ delimiter, or a different closing delimiter, allways at the same level. The ⟨*posTamble:*⟩ allows some fine tuning. The ⟨*scAn:w*⟩ function scans the ⟨*input stream*⟩, and is expected to end on a closing delimiter at the current level by sending \BNVS_scanned_cLose:w, which meaning is ⟨*close:*⟩ followed by \BNVS_scanned_cLose:w. If the ⟨*scAn:w*⟩ will call itself after a block, a comma or a colon if the corresponding branch function is called. This is the normal instruction flow.

```
\cs_new:Npn \BNVS_scan_cLose:N:EBSTN:w #1 #2 #3 #4 #5 #6 {
  \BNVS_scan:EBOCMLSTN:w {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \cs_set:Npn \BNVS_scanned_cLose:nw ##1 {
```

Anyways, branch here.

```
      \BNVS_scanned_cLose:w
    }
    #2
  } {
```

Any branch function, but the two last ones, executes the ⟨*scAn:w*⟩ instructions.

```
    #6
  } {
    #6
  } {
    #6
  } {
    #6
  } {
```

Close branch. End the group, execute ⟨*block:*⟩ instructions and calls the function `\BNVS_scanned_Block:w`. Intermediate groups should be closed appropriately with the right definition of branch functions. The value of `I` is not exported to the containing group.

```
    \BNVS_end_scan_no_I_change:
    #3
  } {
```

Stop branch. Execute the ⟨*stop:*⟩ instructions after the group ends.

```
    \BNVS_error:n { Missing~`#1'. }
    \BNVS_end_scan_no_I_change:
    #4
  } {
```

Execute the ⟨*posTamble:*⟩ instructions.

```
    #5
  } {
```

Execute the ⟨*scAn:w*⟩ instructions.

```
    #6
  }
}
```

GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED

`\c__bnvs_Crl_open_regex` Curly opening delimiter.

(*End of definition for* `\c__bnvs_Crl_open_regex`.)

```
\regex_const:Nn \c__bnvs_Crl_open_regex { \s* \{ %}
  \s* }
```

\BNVS_scan_Crl:EBSTN:Fw `\BNVS_scan_Crl:EBSTN:Fw {⟨prEamble:⟩} {⟨block:⟩} {⟨stop:⟩} {⟨scAn:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan ⟨*Crl*⟩ unit with the help of ⟨*scAn:w*⟩, then execute one of ⟨*block:*⟩ or ⟨*Stop:*⟩ instructions. ⟨*scAn:w*⟩ must end with one of the branch functions `\BNVS_scanned_cLose:w` or `\BNVS_scanned_Stop:`. Otherwise execute ⟨*F:w*⟩ instructions.

```
\group_begin:
\char_set_catcode_group_begin:N (
\char_set_catcode_group_end:N )
\char_set_catcode_other:N \{
\char_set_catcode_other:N \}
\cs_new:Npn \BNVS_scan_Crl:EBSTN:Fw #1 #2 #3 #4 #5 #6 (
  \BNVS_peek_regex_remove_once:NTF \c__bnvs_Crl_open_regex (
    \BNVS_scan_cLose:N:EBSTN:w %{
    } ( #1 ) ( #2 ) ( #3 ) ( #4 ) ( #5 )
  ) (
    #6
  )
)
\group_end:
```

### 1.12.8 Raw units

```
⟨Raw⟩       ::= ⟨Raw_open⟩ ⟨call⟩ ⟨Rnd_close⟩
⟨Raw_open⟩  ::= ⟨spc⟩ '"(' ⟨spc⟩
⟨RawLst⟩    ::= ( ⟨RawRnd⟩ | ⟨RawSqr⟩ | ⟨RawCrl⟩ | ⟨RawOtr⟩ )*
⟨RawRnd⟩    ::= ⟨Rnd_open⟩ ⟨RawLst⟩ ⟨Rnd_close⟩
⟨RawSqr⟩    ::= ⟨Sqr_open⟩ ⟨RawLst⟩ ⟨Sqr_close⟩
⟨RawCrl⟩    ::= ⟨Crl_open⟩ ⟨RawLst⟩ ⟨Crl_close⟩
⟨RawOtr⟩    ::= ⟨characters of category other except delimiters⟩
```

\c__bnvs_open_Raw_regex Raw opening delimiter.

```
\regex_const:Nn \c__bnvs_open_Raw_regex { \s* "\( %)
  \s* }
```

(*End of definition for* `\c__bnvs_open_Raw_regex`.)

\BNVS_scan_Raw_LS:w `\BNVS_scan_Raw_LS:w ⟨input stream⟩`
\BNVS_scan_Raw:nw `\BNVS_scan_Raw:nw {⟨input stream⟩} ⟨input stream⟩`

Scan balanced material taking only delimiters into account. It stops on an unbalanced closing delimiter or in a stop situation. Everything in between is exported as is.

```
\cs_new:Npn \BNVS_scan_Raw:nw #1 {
  \BNVS_xprt_ast:n { #1 }
  \BNVS_scan_Raw_LS:w
}
```

```
\group_begin:
\char_set_catcode_group_begin:N <
\char_set_catcode_group_end:N >
\char_set_catcode_other:N \{
\char_set_catcode_other:N \}
\cs_new:Npn \BNVS_scan_Raw_LS:w  <
```

If we find an opening delimiter.

```
  \BNVS_scan_Rnd:EBSTN:Fw <
    \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >%(
    \BNVS_ast_plc_wrap_mpt:n < ##1 ) >
  > <
    \BNVS_scanned_cLose_set:n <
      \BNVS_end_scan_no_I_change:
      \BNVS_scan_Raw_LS:w
    >
    \BNVS_scanned_Block:w
  > <%(
    \BNVS_error:n < Missing~`)' > %(
    \BNVS_xprt_ast:n < ) >
    \BNVS_scanned_Stop:
  > <
    \BNVS_xprt_ast:n < ( > %)
  > <
    \BNVS_scan_Raw_LS:w
  > <
    \BNVS_scan_Sqr:EBSTN:Fw <
      \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >%[
      \BNVS_ast_plc_wrap_mpt:n < ##1 ] >
    > <
      \BNVS_scanned_cLose_set:n <
        \BNVS_end_scan_no_I_change:
        \BNVS_scan_Raw_LS:w
      >
      \BNVS_scanned_Block:w
    > <%[
      \BNVS_error:n < Missing~`]' > %[
      \BNVS_xprt_ast:n < ] >
      \BNVS_scanned_Stop:
    > <
      \BNVS_xprt_ast:n < [ > %]
    > <
      \BNVS_scan_Raw_LS:w
    > <
      \BNVS_scan_Crl:EBSTN:Fw <
        \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >%{
        \BNVS_ast_plc_wrap_mpt:n < ##1 } >
      > <
        \BNVS_scanned_cLose_set:n <
          \BNVS_end_scan_no_I_change:
          \BNVS_scan_Raw_LS:w
        >
        \BNVS_scanned_Block:w
```

```
      > <%{
        \BNVS_error:n < Missing~`}' > %{
        \BNVS_xprt_ast:n < } >
        \BNVS_scanned_Stop:
      > <
        \BNVS_xprt_ast:n < { > %}
      > <
        \BNVS_scan_Raw_LS:w
      > <
```

The head of the ⟨*input stream*⟩ is not an opening delimiter, we look for a closing delimiter, or a stop situation or finally export one character.

```
        \BNVS_scan_cLose:Fw < \BNVS_scan_Stop:F < \BNVS_scan_Raw:nw > >
> > > >
\group_end:
```

---

`\BNVS_scan_Raw:BS:Fw` `\BNVS_scan_Raw:BS:Fw {⟨scAn:B:w⟩} {⟨scAn:S:⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan balanced material taking only delimiters into account. Starts on a `"(`.

```
\cs_new:Npn \BNVS_scan_Raw:BS:Fw #1 #2 #3 {
  \BNVS_peek_regex_remove_once:NTF \c__bnvs_open_Raw_regex {
    \BNVS_scan_cLose:N:EBSTN:w %(
    ) {
      \BNVS_ast_plc_wrap_pfx:n { \:n { ##1 } }
      \BNVS_xprt_space_set:n { \BNVS_xprt_ast:n { ~ } }
    } { #1 } { #2 } { } {
      \BNVS_scan_Raw_LS:w
    }
  } {
    #3
  }
}
```

### 1.12.9 ⟨*csl(⟨item⟩)*⟩ units

GOBBLED

### 1.12.10 ISR unit

Here is a partial grammar for identifiers,

```
⟨ISR⟩     ::= ( ⟨IS⟩ | ⟨dot_P⟩ ) ⟨R⟩*
⟨IS⟩      ::= ( ⟨I⟩? '!' )? ⟨S⟩
⟨I⟩       ::= ⟨S⟩
⟨S⟩       ::= ⟨alpha⟩ ⟨alnum⟩*
⟨dot_P⟩   ::= '.' ⟨P⟩
⟨P⟩       ::= ⟨digit⟩* ⟨S⟩
⟨R⟩       ::= ⟨dot_P⟩ | ⟨dot_N⟩ | ⟨n⟩
⟨dot_N⟩   ::= '.' ⟨N⟩
⟨n⟩       ::= ⟨Sqr_open⟩ ⟨n_expr⟩ ⟨Sqr_close⟩
⟨n_expr⟩  ::= ⟨blnc⟩
```

and the associate AST :

```
⟨IS⟩*  ::= \IS:w  { ⟨I⟩ } { ⟨S⟩ }
⟨ISR⟩* ::= \ISR:w { ⟨I⟩ } { ⟨S⟩ } { ⟨R⟩* }
⟨R⟩*   ::= ( ⟨P⟩* | ⟨N⟩* | ⟨n⟩* )*
⟨P⟩*   ::= { \P:n { ⟨P⟩ } }
⟨N⟩*   ::= { \N:n { ⟨N⟩ } }
⟨n⟩*   ::= { \n:n { ⟨n_expr⟩* } }
```

\c__bnvs_IKS_regex The start of a qualified dotted name, with three capture groups.

(*End of definition for* `\c__bnvs_IKS_regex`.)

```
\regex_const:Nn \c__bnvs_IKS_regex {
```

1: the optional frame ⟨*id*⟩ with

2: the optional exclamation mark

```
  (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?
```

3: followed by a non empty short name.

```
  ( \ur{c__bnvs_S_regex} )
}
```

---

\BNVS_scanned:B:IKS:w `\BNVS_scanned:B:IKS:w {⟨scAn:B:w⟩} {⟨I⟩} {⟨K⟩} {⟨S⟩} ⟨input stream⟩`

Only called by `\BNVS_scan_IS:B:Fw`. Execute ⟨*scAn:B:w*⟩ instructions, which terminate by `\BNVS_scanned_Block:w`.

```
\cs_new:Npn \BNVS_scanned:B:IKS:w #1 #2 #3 #4 {
  \tl_if_empty:nTF { #3 } {
```

No frame identifier given, use the current value. Store the short name, store in the appropriate variable.

```
    \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
    \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #4 }
  } {
```

Frame identifier given. Proceed as before mutatis mutandis.

```
    \__bnvs_tl_set:cn I { #2 }
    \__bnvs_tl_clear_new:c { S(#2) }
    \__bnvs_tl_set:cn { S(#2) } { #4 }
  }
  \__bnvs_tl_set:cn S { #4 }
```

Create a scan group to manage the ⟨*prefix:w*⟩.

```
  \BNVS_scan:ETN:w {
```

By default we start on a standalone identifier,

```
    \BNVS_ast_plc_wrap:n { ##1 }
    \BNVS_ast_prefix_set:n { \IS:w }
  } {
```

Export ⟨*I*⟩ and ⟨*S*⟩.

```
    \BNVS_tl_use:Nv \BNVS_xprt_grp:n I
    \BNVS_xprt_grp:n { #4 }
  } {
```

Execute the ⟨*scAn:B:w*⟩ instructions including the terminating branch function \BNVS_scanned_Block:w The ⟨*prefix:w*⟩ will possibly change.

```
    \cs_set:Npn \BNVS_scAn:B:w ##1 { #1 }
    \BNVS_scAn:B:w { \BNVS_scanned_Block:w }
  }
}
```

---

\BNVS_scan_IS:B:Fw \BNVS_scan_IS:B:Fw {⟨*scAn:B:w*⟩} {⟨*F:w*⟩} ⟨*input stream*⟩

Scan an ⟨*IS*⟩ unit if any, stores the ⟨*I*⟩, ⟨*S*⟩ between braces, and calls \BNVS_scanned:B:IKS:w that will execute ⟨*scAn:B:w*⟩ instructions (see above). The ⟨*F:w*⟩ instructions are executed when no ⟨*IS*⟩ unit is found.

```
\cs_new:Npn \BNVS_scan_IS:B:Fw #1 #2 {
  \peek_regex_replace_once:NnTF \c__bnvs_IKS_regex {
    \cB\{ \1 \cE\} \cB\{ \2 \cE\} \cB\{ \3 \cE\}
  } {
    \BNVS_scanned:B:IKS:w { #1 }
  } {
```

Choose the false branch.

```
    #2
  }
}
```

---

\BNVS_scanning_PNn:P:w \BNVS_scanning_PNn:P:w {⟨*P*⟩} ⟨*input stream*⟩

Export the ⟨*P*⟩ component then scan another ⟨*PNn*⟩ component.

```
\cs_new:Npn \BNVS_scanning_PNn:P:w #1 {
  \BNVS_xprt_grp:n { \P:n { #1 } }
  \BNVS_scan_PNn:w
}
```

---

\BNVS_scanning_PNn:N:w \BNVS_scanning_PNn:P:w {⟨*N*⟩} ⟨*input stream*⟩

Export the ⟨*N*⟩ component then scan another ⟨*PNn*⟩ component.

```
\cs_new:Npn \BNVS_scanning_PNn:N:w #1 {
```

Normalize ⟨*N*⟩.

```
  \exp_args:Ne \BNVS_xprt_grp:n { \exp_not:N \N:n { \int_eval:n { #1 } } }
  \BNVS_scan_PNn:w
}
```

---

\BNVS_scanning_PNn_n:w \BNVS_scanning_PNn_n:w ⟨*input stream*⟩

Only called by \BNVS_scan_PNn:w.

```
\cs_new:Npn \BNVS_scanning_PNn_n:w {
```

```
  \BNVS_scan_cLose:N:EBSTN:w %[
  ] {
    \BNVS_ast_plc_wrap:n { { \n:n { ##1 } } }
  } {
    \BNVS_scanned_Block:w
  } {
    \BNVS_scanned_Stop:
  } {

```

On cLose branch, scan ⟨*PNn*⟩ after ⟨*n*⟩.

```
    \BNVS_scanned_cLose_set:n { \BNVS_end_scan_no_I_change: \BNVS_scan_PNn:w }
  } {
    \BNVS_scan_n_expr_LS:w
  }
}
```

---

`\BNVS_scan_PNn:w` `\BNVS_scan_PNn:w` ⟨*input stream*⟩

Scan all the individual ⟨*R*⟩ components, exporting them one after the other, then execute `\BNVS_scanned_Block:w` branch function. Calls `\BNVS_scanning_PNn:P:w`, `\BNVS_scanning_PNn:N:w` or `\BNVS_scanning_PNn_n:w`.

```
\cs_new:Npn \BNVS_scan_PNn:w {
  \peek_charcode_remove:NTF . {
    \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
      \BNVS_scanning_PNn:P:w
    } {
      \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
        \BNVS_scanning_PNn:N:w
      } {
```

Terminates the block and reinject the dot '.'.

```
        \BNVS_scanned_Block:w .
      }
    }
  } {
    \BNVS_peek_regex_remove_once:NTF \c__bnvs_Sqr_open_regex {
      \BNVS_scanning_PNn_n:w
    } {
```

Also terminates the block.

```
      \BNVS_scanned_Block:w
    }
  }
}
```

---

`\BNVS_scanning_R:B:P:w` `\BNVS_scanning_R:B:P:w` {⟨*scAn_B:w*⟩} {⟨*P*⟩} ⟨*input stream*⟩

Only called by `\BNVS_scan_R:B:w` when a heading ⟨*P*⟩ has just been scanned. Calls `\BNVS_scan_PNn:w`.

```
\cs_new:Npn \BNVS_scanning_R:B:P:w #1 #2 {
```

New scan group to gather the ⟨*R*⟩ relative path.

```
  \BNVS_scan:ETN:w {
```

```
      \BNVS_ast_plc_wrap_grp:n { ##1 }
      \BNVS_xprt_grp:n { \P:n { #2 } }
    } {
      \BNVS_scanned_Block_set:n {
        \BNVS_end_scan_no_I_change:
        #1
      }
    } {
      \BNVS_scan_PNn:w
    }
}
```

---

\BNVS_scanning_R:B:N:w `\BNVS_scanning_R:B:N:w {⟨scAn_B:w⟩} {⟨N⟩} ⟨input stream⟩`

---

Only called by `\BNVS_scan_R:B:w` when a heading ⟨*N*⟩ has just been scanned. Calls `\BNVS_scan_PNn:w`.

```
\cs_new:Npn \BNVS_scanning_R:B:N:w #1 #2 {
```

New scan group to gather the ⟨*R*⟩ relative path.

```
  \BNVS_scan:ETN:w {
    \BNVS_ast_plc_wrap_grp:n { ##1 }
    \exp_args:Ne \BNVS_xprt_grp:n {
```

Normalize ⟨*N*⟩.

```
      \exp_not:N \N:n { \int_eval:n { #2 } }
    }
  } {
    \BNVS_scanned_Block_set:n {
      \BNVS_end_scan_no_I_change:
      #1
    }
  } {
    \BNVS_scan_PNn:w
  }
}
```

---

\BNVS_scanning_R_n:B:w `\BNVS_scanning_R_n:B:w {⟨scAn_B:w⟩} ⟨input stream⟩`

---

Only called by `\BNVS_scan_R:B:w` when a heading '[' has just been scanned. Terminates by `\BNVS_scan_PNn:w` to scan another component. See above for ⟨*scAn:B:w*⟩.

```
\cs_new:Npn \BNVS_scanning_R_n:B:w #1 {
```

New scan group to gather the ⟨*R*⟩ relative path.

```
  \BNVS_scan:ETN:w {
    \BNVS_ast_plc_wrap_grp:n { ##1 }
  } {
```

On block branch, close the scan group and execute the ⟨*scAn_B:w*⟩ instructions.

```
    \BNVS_scanned_Block_set:n {
      \BNVS_end_scan_no_I_change:
      #1
    }
  } {
```

Export material up to the closing square bracket as one single argument group.

```
    \BNVS_scan_cLose:N:EBSTN:w %[
    ] {
      \BNVS_ast_plc_wrap:n { { \n:n { ##1 } } }
    } {
      \BNVS_scanned_Block:w
    } {
      \BNVS_scanned_Stop:
    } {
```

On close branch, scan ⟨*PNn*⟩ after ⟨*n*⟩.

```
      \BNVS_scanned_cLose_set:n { \BNVS_end_scan_no_I_change: \BNVS_scan_PNn:w }
    } {
      \BNVS_scan_n_expr_LS:w
    }
  }
}
```

---

\BNVS_scan_R:B:w `\BNVS_scan_R:B:w {⟨scAn:B:w⟩} ⟨input stream⟩`

Scan a ⟨*R*⟩ component and execute ⟨*scAn_B:w*⟩ instructions. The scanning stops exactly when some regular expression matches, or when squared material is scanned. The function \BNVS_scan_R:B:w calls

- \BNVS_scanning_R:B:P:w,
- \BNVS_scanning_R:B:N:w or
- \BNVS_scanning_R_n:B:w

with ⟨*scAn:B:w*⟩ as argument.

```
\cs_new:Npn \BNVS_scan_R:B:w #1 {
  \peek_charcode_remove:NTF . {
    \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
      \BNVS_scanning_R:B:P:w { #1 }
    } {
      \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
        \BNVS_scanning_R:B:N:w { #1 }
      } {
```

Terminates the block and reinject the dot.

```
        \BNVS_xprt_grp:n { }
        #1
        .
      }
    }
  } {
    \BNVS_peek_regex_remove_once:NTF \c__bnvs_Sqr_open_regex {
      \BNVS_scanning_R_n:B:w { #1 }
    } {
```

Terminates the block too.

```
      \BNVS_xprt_grp:n { }
      #1
```

```
    }
  }
}

\tl_new:N \l__bnvs_other_X_tl
\tl_set:Ne \l__bnvs_other_X_tl { \tl_to_str:n { X } }
```

`\BNVS_scanned_dot:B:P:w` `\BNVS_scanned_dot:B:P:w {⟨scAn:B:w⟩} {⟨P⟩} ⟨input stream⟩`
`\BNVS_scanned_dot:n:B:P:w` `\BNVS_scanned_dot:B:P:w {⟨before⟩} {⟨scAn:B:w⟩} {⟨P⟩} ⟨input stream⟩`

Only called by `\BNVS_scan_dot_P:B:Fw`. The second functions calls rhe first one with an empty ⟨*before*⟩.

```
\cs_new:Npn \BNVS_scanned_dot:n:B:P:w #1 #2 #3 {
```

No frame identifier given, no short name given.

```
  \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {
```

It defaults to `X`.

```
    \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
    \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
  }
  \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I) }
```

Create a scan group to manage the ⟨*prefix:w*⟩. It will be closed by a call to `\BNVS_scanned_Block:w`.

```
  \BNVS_scan:ETN:w {
```

By default we start on a standalone identifier,

```
    \BNVS_ast_prefix_set:n { \ISR:w }
    \exp_args:Ne \BNVS_ast_plc_wrap_mpt:n {
      { \__bnvs_tl_use:c I }
      { \__bnvs_tl_use:c S }
      { \exp_not:n { ##1 } }
    }
    \BNVS_xprt_ast:n { { \P:n { #3 } } }
  } {
    \BNVS_scanned_Block_wrap:n:B { #1 } { #2 }
```

Create a scan group to manage the ⟨*prefix:w*⟩.

```
  } {
```

Find the next components.

```
    \BNVS_scan_PNn:w
  }
}

\cs_new:Npn \BNVS_scanned_dot:B:P:w {
  \BNVS_scanned_dot:n:B:P:w {}
}
```

`\BNVS_scan_dot_P:B:Fw` `\BNVS_scan_dot_P:B:Fw {⟨scAn:B:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*dot_P*⟩ unit and execute ⟨*scAn:B:w*⟩ instructions that must end with `\BNVS_scanned_Block:w`. The ⟨*F:w*⟩ instructions are executed when no ⟨*dot_P*⟩ unit is found.

```
\cs_new:Npn \BNVS_scan_dot_P:B:Fw #1 #2 {
  \peek_charcode_remove:NTF . {
    \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
      \BNVS_scanned_dot:B:P:w { #1 }
    } {
```

Choose the false branch and reinject the dot.

```
      #2
      .
    }
  } {
```

Choose also the false branch.

```
    #2
  }
}
```

---

`\BNVS_scanned_dot:n:B:N:w` `\BNVS_scanned_dot:B:N:w` `\BNVS_scanned_dot:n:B:N:w {⟨before⟩} {⟨scAn:B:w⟩} {⟨N⟩} ⟨input stream⟩`
`\BNVS_scanned_dot:B:N:w {⟨scAn:B:w⟩} {⟨N⟩} ⟨input stream⟩`

---

Only called by `\BNVS_scan_dot_N:B:Fw`.

```
\cs_new:Npn \BNVS_scanned_dot:n:B:N:w #1 #2 #3 {
```

No frame identifier given, no short name given.

```
  \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {
```

It defaults to `X`.

```
    \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
    \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
  }
  \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I) }
```

Create a scan group to manage the ⟨*prefix:w*⟩. It will be closed by a call to `\BNVS_scanned_Block:w`.

```
  \BNVS_scan:ETN:w {
```

By default we start on a standalone identifier,

```
    \BNVS_ast_prefix_set:n { \ISR:w }
    \exp_args:Ne \BNVS_ast_plc_wrap_mpt:n {
      { \__bnvs_tl_use:c I }
      { \__bnvs_tl_use:c S }
      { \exp_not:n { ##1 } }
    }
    \exp_args:Ne \BNVS_xprt_ast:n {
      { \exp_not:N \N:n { \int_eval:n { #3 } } }
    }
  } {
    \BNVS_scanned_Block_wrap:n:B { #1 } { #2 }
```

Create a scan group to manage the ⟨*prefix:w*⟩.

```
  } {
```

Find the next components.

```
    \BNVS_scan_PNn:w
  }
}

\cs_new:Npn \BNVS_scanned_dot:B:N:w {
  \BNVS_scanned_dot:n:B:N:w { }
}
```

Create a scan group to manage the ⟨*prefix:w*⟩.

---

\BNVS_scan_dot_N:B:Fw `\BNVS_scan_dot_N:B:Fw {⟨scAn:B:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*dot_N*⟩ unit and execute ⟨*scAn:B:w*⟩ instructions that must end with `\BNVS_scanned_Block:w`. The ⟨*F:w*⟩ instructions are executed when no ⟨*dot_N*⟩ unit is found.

```
\cs_new:Npn \BNVS_scan_dot_N:B:Fw #1 #2 {
  \peek_charcode_remove:NTF . {
    \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
      \BNVS_scanned_dot:B:N:w { #1 }
    } {
```

Choose the false branch and reinject the dot.

```
      #2
      .
    }
  } {
```

Choose also the false branch.

```
    #2
  }
}
```

---

\BNVS_scan_ISR:B:Fw `\BNVS_scan_ISR:B:Fw {⟨scAn:b:⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan an ⟨*ISR*⟩ unit if any, ⟨*scAn:b:*⟩ instructions terminated by a call to the branch function `\BNVS_scanned_Block:w`. The ⟨*F:w*⟩ instructions are executed when no ⟨*ISR*⟩ unit is found.

```
\cs_new:Npn \BNVS_scan_ISR:B:Fw #1 #2 {
  \BNVS_scan_IS:B:Fw {
    \BNVS_scan_R:B:w { #1 }
  } {
    #2
  }
}
```

### 1.12.11 Assignment units

Here is the grammar

```
⟨ISRMOR⟩   ::= ⟨ISR⟩ ⟨MOR⟩
⟨MOR⟩      ::= ⟨ModOpEq⟩ ⟨Rhs⟩
⟨ModOpEq⟩  ::= ⟨spc⟩ ( ⟨Mod⟩ ⟨Op⟩ '=' ) ⟨spc⟩
⟨Mod⟩      ::= '' | '?'
⟨Op⟩       ::= '' | '+' | '-' | '/' | '*'
⟨Rhs⟩      ::= ⟨see below⟩
```

with the associate AST

```
⟨ISRMOR⟩* ::= \ISRMOR:w { ⟨I⟩ } { ⟨S⟩ } { ⟨R⟩* } { ⟨Mod⟩ } { ⟨Op⟩ } {
↪  ⟨Rhs⟩* }
```

`\c__bnvs_ModOpEq_regex` Assignment operators, two capture groups.

(*End of definition for* `\c__bnvs_ModOpEq_regex`.)

```
\regex_const:Nn \c__bnvs_ModOpEq_regex {
  \s*
```

1: an optional modifier.

```
  ( \? )?
```

2: an optional operator.

```
  ( [-+/*] )?

  =\s*
}
```

`\BNVS_scanned_ModOpEq:MLS:nnw` `\BNVS_scanned_ModOpEq:MLS:nnw {⟨comMa:w⟩} {⟨cLose:w⟩} {⟨Stop:⟩} {⟨Mod⟩} {⟨Op⟩} ⟨input stream⟩`

Only called by `\BNVS_scan_MOR:MLS:Fw`. Calls `\BNVS_scan_MLS:w`.

```
\cs_new:Npn \BNVS_scanned_ModOpEq:MLS:nnw #1 #2 #3 #4 #5 {
  \BNVS_xprt_grp:n { #4 }
  \BNVS_xprt_grp:n { #5 }
  \BNVS_scan:EBOCMLSTN:w {
    \BNVS_ast_plc_wrap_grp:n { ##1 }
  } { \BNVS_error:n { Unexpected } \BNVS_scanned_cLose:w }
    { \BNVS_error:n { Unexpected } \BNVS_scanned_cLose:w }
    { \BNVS_error:n { Unexpected } \BNVS_scanned_cLose:w }
    { \BNVS_end_scan_no_I_change: #1 }
    { \BNVS_end_scan_no_I_change: #2 }
    { \BNVS_end_scan_no_I_change: #3 }
    { }
    { \BNVS_scan_MLS:w }
}
```

\BNVS_scan_MOR:MLS:Fw `\BNVS_scan_MOR:MLS:Fw {⟨comMa:w⟩} {⟨cLose:w⟩} {⟨Stop:⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*ModOpEq*⟩ unit, if any, and execute one of ⟨*comMa:w*⟩, ⟨*cLose:w*⟩ or ⟨*Stop:*⟩ instructions, otherwise execute ⟨*F:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scan_MOR:MLS:Fw #1 #2 #3 #4 {
  \peek_regex_replace_once:NnTF \c__bnvs_ModOpEq_regex {
    \cB\{ \1 \cE\} \cB\{ \2 \cE\}
  } {
    \BNVS_ast_prefix_will_set:n { \ISRMOR:w }
    \BNVS_scanned_ModOpEq:MLS:nnw { #1 } { #2 } { #3 }
  } {
    #4
  }
}
```

\BNVS_scan_ISRMOR:MLS:Fw `\BNVS_scan_ISRMOR:MLS:Fw {⟨comMa:w⟩} {⟨cLose:w⟩} {⟨Stop:⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*ISRMOR*⟩ unit, if any, and execute one of ⟨*comMa:w*⟩, ⟨*cLose:w*⟩ or ⟨*Stop:*⟩ instructions, otherwise execute ⟨*F:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scan_ISRMOR:MLS:Fw #1 #2 #3 #4 {
  \BNVS_scan_ISR:B:Fw {
    \BNVS_scan_MOR:MLS:Fw {
      \BNVS_ast_prefix_will_set:n { \ISRMOR }
      #1
    } {
      \BNVS_ast_prefix_will_set:n { \ISRMOR }
      #2
    } {
      \BNVS_ast_prefix_will_set:n { \ISRMOR }
      #3
    } { #4 }
  } {
    #4
  }
}
```

### 1.12.12 Call units

Here is the grammar

⟨*ISRC*⟩ ::= ⟨*ISR*⟩ ⟨*call*⟩
⟨*call*⟩ ::= ⟨*Rnd_open*⟩ ⟨*blnc*⟩ ⟨*Rnd_close*⟩

with the associate AST

⟨*ISRC*⟩$^\star$ ::= \ISRC:w { ⟨*I*⟩ } { ⟨*S*⟩ } { ⟨*R*⟩$^\star$ } { ⟨*Raw*⟩$^\star$ }

\BNVS_scan_call:B:w `\BNVS_scan_call:B:Fw {⟨scAn:B:w⟩} ⟨input stream⟩`

Scan a ⟨*call*⟩ unit, if any, and execute ⟨*scAn:B:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scan_call:B:w #1 {
  \BNVS_scan_Rnd:EBSTN:Fw { } { #1 } {
    \BNVS_scanned_Stop:
  } { } {
    \BNVS_scan_Raw_LS:w
  } { #1 }
}
```

\BNVS_scan_ISRC:B:Fw `\BNVS_scan_ISRC:B:Fw {⟨scAn:B:w⟩}  {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*ISRC*⟩ unit, if any, and execute ⟨*scAn:B:w*⟩ instructions, otherwise execute ⟨*F:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scan_ISRC:B:Fw #1 {
  \BNVS_scan_ISR:B:Fw {
    \BNVS_ast_prefix_will_set:n { \ISRC:w }
    \BNVS_scan_call:B:w { #1 }
  }
```

There must be absolutely no code here. No hook is allowed either.

```
}
```